LAB SHEET-6

NAME: GADDAM PRANAVI REG NO.: 23BCE20060

1. Use MongoDB to implement the following DB operations

   1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

```
Current Mongosh Log ID: 69b1753a8b4d8a77307c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-03-11T19:20:43.753+05:30: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
------

test> use vehicles
switched to db vehicles
vehicles>
```

   2. Write a MongoDB query to display all the databases.

```
vehicles> show dbs
admin     40.00 KiB
config   108.00 KiB
local     40.00 KiB
vehicles>
```

   3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

```
vehicles> db.createCollection("two_wheelers", {capped:true, size:100000})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
vehicles>
```

   4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Honda Shine",
| model: "gear",
| category: "125cc",
| colors_available: ["red","black","blue"],
| manufacturer: "Honda",
| performance: 8,
| timestamp: new Date("2022-05-01"),
| price: 90000
| },
|
| {
| bike_name: "TVS Jupiter",
| model: "gearless",
| category: "110cc",
| colors_available: ["silver","blue","black"],
| manufacturer: "TVS",
| performance: 7,
| timestamp: new Date("2021-04-15"),
| price: 85000
| },
|
| {
| bike_name: "Yamaha R15",
| model: "gear",
| category: "150cc",
| colors_available: ["sport red","black"],
| manufacturer: "Yamaha",
| performance: 9,
| timestamp: new Date("2023-01-10"),
| price: 180000
| },
|
```

```
|
| {
| bike_name: "Bajaj Pulsar",
| model: "gear",
| category: "150cc",
| colors_available: ["black","blue"],
| manufacturer: "Bajaj",
| performance: 8,
| timestamp: new Date("2020-08-20"),
| price: 120000
| },
|
| {
| bike_name: "Suzuki Access",
| model: "gearless",
| category: "125cc",
| colors_available: ["white","red"],
| manufacturer: "Suzuki",
| performance: 7,
| timestamp: new Date("2022-02-05"),
| price: 95000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b1775d8b4d8a77307c2907'),
    '1': ObjectId('69b1775d8b4d8a77307c2908'),
    '2': ObjectId('69b1775d8b4d8a77307c2909'),
    '3': ObjectId('69b1775d8b4d8a77307c290a'),
    '4': ObjectId('69b1775d8b4d8a77307c290b')
  }
}
vehicles>
```

5. Add 5 four-wheeler details to the collection named 'four_wheelers'. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

```
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Maruti Swift",
| model: "own",
| category: "car",
| variants: ["vxi","zxi","petrol"],
| manufacturer: "Maruti",
| performance: 8,
| timestamp: new Date("2022-01-10"),
| price: 700000
| },
|
| {
| vehicle_name: "Tata Ace",
| model: "commercial",
| category: "mini truck",
| variants: ["diesel"],
| manufacturer: "Tata",
| performance: 7,
| timestamp: new Date("2021-06-15"),
| price: 500000
| },
|
| {
| vehicle_name: "Ashok Leyland Bus",
| model: "commercial",
| category: "bus",
| variants: ["diesel"],
| manufacturer: "Ashok Leyland",
| performance: 8,
| timestamp: new Date("2020-09-25"),
| price: 2500000
| },
|
| {
| vehicle_name: "Mahindra Bolero",
| model: "own",
| category: "car",
| variants: ["diesel","zxi"],
| manufacturer: "Mahindra",
| performance: 7,
| timestamp: new Date("2023-02-11"),
| price: 950000
| },
|
| {
| vehicle_name: "Volvo Container Truck",
| model: "commercial",
| category: "containers",
| variants: ["diesel"],
| manufacturer: "Volvo",
| performance: 9,
| timestamp: new Date("2021-11-01"),
| price: 3500000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b178178b4d8a77307c290c'),
    '1': ObjectId('69b178178b4d8a77307c290d'),
    '2': ObjectId('69b178178b4d8a77307c290e'),
    '3': ObjectId('69b178178b4d8a77307c290f'),
    '4': ObjectId('69b178178b4d8a77307c2910')
  }
}
vehicles>
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Two Wheelers

```
vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69b1775d8b4d8a77307c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2022-05-01T00:00:00.000Z'),
    price: 90000
  },
  {
    _id: ObjectId('69b1775d8b4d8a77307c2908'),
    bike_name: 'TVS Jupiter',
    model: 'gearless',
    category: '110cc',
    colors_available: [ 'silver', 'blue', 'black' ],
    manufacturer: 'TVS',
    performance: 7,
    timestamp: ISODate('2021-04-15T00:00:00.000Z'),
    price: 85000
  },
  {
    _id: ObjectId('69b1775d8b4d8a77307c2909'),
    bike_name: 'Yamaha R15',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'sport red', 'black' ],
    manufacturer: 'Yamaha',
    performance: 9,
    timestamp: ISODate('2023-01-10T00:00:00.000Z'),
    price: 180000
  },
```

```
  {
    _id: ObjectId('69b1775d8b4d8a77307c290a'),
    bike_name: 'Bajaj Pulsar',
    model: 'gear',
    category: '150cc',
    colors_available: [ 'black', 'blue' ],
    manufacturer: 'Bajaj',
    performance: 8,
    timestamp: ISODate('2020-08-20T00:00:00.000Z'),
    price: 120000
  },
  {
    _id: ObjectId('69b1775d8b4d8a77307c290b'),
    bike_name: 'Suzuki Access',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'white', 'red' ],
    manufacturer: 'Suzuki',
    performance: 7,
    timestamp: ISODate('2022-02-05T00:00:00.000Z'),
    price: 95000
  }
]
vehicles>
```

Four Wheelers

```
vehicles> db.four_wheelers.find()
[
  {
    _id: ObjectId('69b178178b4d8a77307c290c'),
    vehicle_name: 'Maruti Swift',
    model: 'own',
    category: 'car',
    variants: [ 'vxi', 'zxi', 'petrol' ],
    manufacturer: 'Maruti',
    performance: 8,
    timestamp: ISODate('2022-01-10T00:00:00.000Z'),
    price: 700000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c290d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    price: 500000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: ISODate('2020-09-25T00:00:00.000Z'),
    price: 2500000
  },
```

```
  {
    _id: ObjectId('69b178178b4d8a77307c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Mahindra',
    performance: 7,
    timestamp: ISODate('2023-02-11T00:00:00.000Z'),
    price: 950000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c2910'),
    vehicle_name: 'Volvo Container Truck',
    model: 'commercial',
    category: 'containers',
    variants: [ 'diesel' ],
    manufacturer: 'Volvo',
    performance: 9,
    timestamp: ISODate('2021-11-01T00:00:00.000Z'),
    price: 3500000
  }
]
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

```
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Honda Shine', price: 90000 },
  { bike_name: 'TVS Jupiter', price: 85000 },
  { bike_name: 'Yamaha R15', price: 180000 },
  { bike_name: 'Bajaj Pulsar', price: 120000 },
  { bike_name: 'Suzuki Access', price: 95000 }
]
```

```
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Maruti Swift', price: 700000 },
  { vehicle_name: 'Tata Ace', price: 500000 },
  { vehicle_name: 'Ashok Leyland Bus', price: 2500000 },
  { vehicle_name: 'Mahindra Bolero', price: 950000 },
  { vehicle_name: 'Volvo Container Truck', price: 3500000 }
]
```

8. Write a MongoDB query to display two_wheelers from a particular company

```
vehicles> db.two_wheelers.find({manufacturer:"Honda"})
[
  {
    _id: ObjectId('69b1775d8b4d8a77307c2907'),
    bike_name: 'Honda Shine',
    model: 'gear',
    category: '125cc',
    colors_available: [ 'red', 'black', 'blue' ],
    manufacturer: 'Honda',
    performance: 8,
    timestamp: ISODate('2022-05-01T00:00:00.000Z'),
    price: 90000
  }
]
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

```
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69b178178b4d8a77307c290d'),
    vehicle_name: 'Tata Ace',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Tata',
    performance: 7,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    price: 500000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c290e'),
    vehicle_name: 'Ashok Leyland Bus',
    model: 'commercial',
    category: 'bus',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 8,
    timestamp: ISODate('2020-09-25T00:00:00.000Z'),
    price: 2500000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c290f'),
    vehicle_name: 'Mahindra Bolero',
    model: 'own',
    category: 'car',
    variants: [ 'diesel', 'zxi' ],
    manufacturer: 'Mahindra',
    performance: 7,
    timestamp: ISODate('2023-02-11T00:00:00.000Z'),
    price: 950000
  },
  {
    _id: ObjectId('69b178178b4d8a77307c2910'),
    vehicle_name: 'Volvo Container Truck',
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

```
vehicles> db.two_wheelers.find(
| {performance:{$gt:5}},
| {bike_name:1, category:1, manufacturer:1, _id:0}
| )
[
  { bike_name: 'Honda Shine', category: '125cc', manufacturer: 'Honda' },
  { bike_name: 'TVS Jupiter', category: '110cc', manufacturer: 'TVS' },
  { bike_name: 'Yamaha R15', category: '150cc', manufacturer: 'Yamaha' },
  { bike_name: 'Bajaj Pulsar', category: '150cc', manufacturer: 'Bajaj' },
  {
    bike_name: 'Suzuki Access',
    category: '125cc',
    manufacturer: 'Suzuki'
  }
]
```

```
vehicles> db.four_wheelers.find(
| {performance:{$gt:5}},
| {vehicle_name:1, category:1, manufacturer:1, _id:0}
| )
[
  {
    vehicle_name: 'Maruti Swift',
    category: 'car',
    manufacturer: 'Maruti'
  },
  {
    vehicle_name: 'Tata Ace',
    category: 'mini truck',
    manufacturer: 'Tata'
  },
  {
    vehicle_name: 'Ashok Leyland Bus',
    category: 'bus',
    manufacturer: 'Ashok Leyland'
  },
  {
    vehicle_name: 'Mahindra Bolero',
    category: 'car',
    manufacturer: 'Mahindra'
  },
  {
    vehicle_name: 'Volvo Container Truck',
    category: 'containers',
    manufacturer: 'Volvo'
  }
]
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called 'animal' and *write* a MongoDB query to select database as 'animal'.

```
admin> use animal
switched to db animal
animal>
```

2. Write a MongoDB query to display all the databases.

```
animal> show dbs
admin      40.00 KiB
config     96.00 KiB
local      40.00 KiB
vehicles   80.00 KiB
animal>
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

```
animal> db.createCollection("wild_animals", {capped:true, size:100000})
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal>
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.wild_animals.insertMany([
| {
| animal_name: "Lion",
| nature: "harm",
| favorite_foods: ["meat","deer"],
| care_taker_name: "Ravi",
| life_span: 15,
| timestamp: new Date(),
| expenses: 50000
| },
|
| {
| animal_name: "Tiger",
| nature: "harm",
| favorite_foods: ["meat","rabbit"],
| care_taker_name: "Suresh",
| life_span: 18,
| timestamp: new Date(),
| expenses: 60000
| },
|
| {
| animal_name: "Elephant",
| nature: "harmless",
| favorite_foods: ["grass","fruits"],
| care_taker_name: "Ramu",
| life_span: 60,
| timestamp: new Date(),
| expenses: 80000
| },
```

```
| {
| animal_name: "Leopard",
| nature: "harm",
| favorite_foods: ["meat"],
| care_taker_name: "Ravi",
| life_span: 20,
| timestamp: new Date(),
| expenses: 55000
| },
|
| {
| animal_name: "Giraffe",
| nature: "harmless",
| favorite_foods: ["leaves"],
| care_taker_name: "Kiran",
| life_span: 25,
| timestamp: new Date(),
| expenses: 45000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b17c3d8b4d8a77307c2911'),
    '1': ObjectId('69b17c3d8b4d8a77307c2912'),
    '2': ObjectId('69b17c3d8b4d8a77307c2913'),
    '3': ObjectId('69b17c3d8b4d8a77307c2914'),
    '4': ObjectId('69b17c3d8b4d8a77307c2915')
  }
}
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

```
animal> db.domestic_animals.insertMany([
| {
| animal_name: "Dog",
| gender: "male",
| favorite_foods: ["meat","bones"],
| animal_petname: "Tommy",
| life_span: 12,
| timestamp: new Date(),
| expenses: 5000
| },
|
| {
| animal_name: "Cat",
| gender: "female",
| favorite_foods: ["fish","milk"],
| animal_petname: "Kitty",
| life_span: 10,
| timestamp: new Date(),
| expenses: 4000
| },
|
| {
| animal_name: "Cow",
| gender: "female",
| favorite_foods: ["grass"],
| animal_petname: "Lakshmi",
| life_span: 20,
| timestamp: new Date(),
| expenses: 7000
| },
|
| {
| animal_name: "Goat",
| gender: "male",
| favorite_foods: ["grass","leaves"],
| animal_petname: "Bunny",
| life_span: 15,
| timestamp: new Date(),
| expenses: 3500
```

```
| {
| animal_name: "Rabbit",
| gender: "female",
| favorite_foods: ["carrot","grass"],
| animal_petname: "Snow",
| life_span: 8,
| timestamp: new Date(),
| expenses: 2000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b17c958b4d8a77307c2916'),
    '1': ObjectId('69b17c958b4d8a77307c2917'),
    '2': ObjectId('69b17c958b4d8a77307c2918'),
    '3': ObjectId('69b17c958b4d8a77307c2919'),
    '4': ObjectId('69b17c958b4d8a77307c291a')
  }
}
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

Wild Animals:

```
animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat', 'rabbit' ],
    care_taker_name: 'Suresh',
    life_span: 18,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Ramu',
    life_span: 60,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 80000
  },
```

```
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2914'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 55000
  },
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2915'),
    animal_name: 'Giraffe',
    nature: 'harmless',
    favorite_foods: [ 'leaves' ],
    care_taker_name: 'Kiran',
    life_span: 25,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 45000
  }
]
```

Domestic Animals:

```
animal> db.domestic_animals.find()
[
  {
    _id: ObjectId('69b17c958b4d8a77307c2916'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'bones' ],
    animal_petname: 'Tommy',
    life_span: 12,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 5000
  },
  {
    _id: ObjectId('69b17c958b4d8a77307c2917'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 10,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 4000
  },
  {
    _id: ObjectId('69b17c958b4d8a77307c2918'),
    animal_name: 'Cow',
    gender: 'female',
    favorite_foods: [ 'grass' ],
    animal_petname: 'Lakshmi',
    life_span: 20,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 7000
  },
```

```
    _id: ObjectId('69b17c958b4d8a77307c2919'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'grass', 'leaves' ],
    animal_petname: 'Bunny',
    life_span: 15,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 3500
  },
  {
    _id: ObjectId('69b17c958b4d8a77307c291a'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrot', 'grass' ],
    animal_petname: 'Snow',
    life_span: 8,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 2000
  }
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

```
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 80000 },
  { animal_name: 'Leopard', expenses: 55000 },
  { animal_name: 'Giraffe', expenses: 45000 }
]
```

```
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 5000 },
  { animal_name: 'Cat', expenses: 4000 },
  { animal_name: 'Cow', expenses: 7000 },
  { animal_name: 'Goat', expenses: 3500 },
  { animal_name: 'Rabbit', expenses: 2000 }
]
animal>
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

```
animal> db.domestic_animals.find({life_span:10})
[
  {
    _id: ObjectId('69b17c958b4d8a77307c2917'),
    animal_name: 'Cat',
    gender: 'female',
    favorite_foods: [ 'fish', 'milk' ],
    animal_petname: 'Kitty',
    life_span: 10,
    timestamp: ISODate('2026-03-11T14:30:45.457Z'),
    expenses: 4000
  }
]
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

```
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 15,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69b17c3d8b4d8a77307c2914'),
    animal_name: 'Leopard',
    nature: 'harm',
    favorite_foods: [ 'meat' ],
    care_taker_name: 'Ravi',
    life_span: 20,
    timestamp: ISODate('2026-03-11T14:29:17.608Z'),
    expenses: 55000
  }
]
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

```
animal> db.wild_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  {
    animal_name: 'Tiger',
    favorite_foods: [ 'meat', 'rabbit' ],
    expenses: 60000
  },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits' ],
    expenses: 80000
  },
  { animal_name: 'Leopard', favorite_foods: [ 'meat' ], expenses: 55000 },
  {
    animal_name: 'Giraffe',
    favorite_foods: [ 'leaves' ],
    expenses: 45000
  }
]
```

```
animal> db.domestic_animals.find(
| {life_span:{$gt:5}},
| {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Dog',
    favorite_foods: [ 'meat', 'bones' ],
    expenses: 5000
  },
  {
    animal_name: 'Cat',
    favorite_foods: [ 'fish', 'milk' ],
    expenses: 4000
  },
  { animal_name: 'Cow', favorite_foods: [ 'grass' ], expenses: 7000 },
  {
    animal_name: 'Goat',
    favorite_foods: [ 'grass', 'leaves' ],
    expenses: 3500
  },
  {
    animal_name: 'Rabbit',
    favorite_foods: [ 'carrot', 'grass' ],
    expenses: 2000
  }
]
```